HTL Leonding
next level

HÖHERE TECHNISCHE
BUNDESLEHRANSTALT
LEONDING

KI-gestützte News-Distribution: Automatisierte Content-Aufbereitung und modulare Bereitstellung von Short-Form-Inhalten

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für Informatik

Eingereicht von:
David Windischbauer
Stefan Schachner

Betreuer:
TODO Betreuer:in

Projektpartner:
Werbeberg

Leonding, 2. April 2027

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 2. April 2027 David Windischbauer & Stefan Schachner

Abstract

News websites publish long articles, while a growing share of readers, especially younger ones, consume news as short, vertically scrolling content on social media platforms. This thesis describes the development of wb-newsfeed, a system that turns regular news articles into short-form content and delivers it as a swipeable feed that can be embedded into existing websites. The project was carried out together with Werbeberg as project partner.



A backend based on Fastify, PostgreSQL and a BullMQ job queue processes every new or changed article asynchronously. A locally hosted large language model (Qwen3 via Ollama) generates a teaser of at most three sentences, three key points and up to two topic tags. A separate image server creates a cover image in portrait format with the diffusion model Z-Image-Turbo. Every generated result is stored as an immutable version together with the model and its sampling parameters, so that results can be reproduced, compared and restored. On the client side, the complete feed is shipped as a reusable Vue and Nuxt package. It loads articles page by page, renders only the cards close to the visible one, and sorts the feed for each reader based on dwell time, likes and the content similarity of articles, without requiring a user account. An admin interface allows editors to review, edit and regenerate the generated content.

Zusammenfassung

Nachrichtenseiten veröffentlichen lange Artikel, während ein wachsender Teil des Publikums, vor allem jüngere Menschen, Nachrichten als kurze, vertikal scrollbare Inhalte auf sozialen Plattformen konsumiert. Diese Arbeit beschreibt die Entwicklung von wb-newsfeed, einem System, das gewöhnliche Nachrichtenartikel automatisch in Short-Form-Inhalte umwandelt und sie als wischbaren Feed bereitstellt, der sich in bestehende Websites einbauen lässt. Projektpartner der Arbeit ist Werbeberg.



Ein Backend auf Basis von Fastify, PostgreSQL und einer BullMQ-Warteschlange verarbeitet jeden neuen oder geänderten Artikel asynchron. Ein lokal betriebenes Sprachmodell (Qwen3 über Ollama) erzeugt daraus einen Teaser mit höchstens drei Sätzen, drei Kernpunkte und bis zu zwei Themen-Tags. Ein eigener Bildserver erstellt mit dem Diffusionsmodell Z-Image-Turbo ein Titelbild im Hochformat. Jedes generierte Ergebnis wird als unveränderliche Version mit Modell und Parametern gespeichert, sodass Ergebnisse nachvollziehbar, vergleichbar und wiederherstellbar bleiben. Auf der Client-Seite steht der komplette Feed als wiederverwendbares Vue- und Nuxt-Package zur Verfügung. Er lädt Artikel seitenweise, rendert nur die Karten in der Nähe der sichtbaren und sortiert den Feed für jede:n Leser:in anhand von Verweildauer, Likes und inhaltlicher Ähnlichkeit der Artikel, ohne dass ein Benutzerkonto nötig ist. Über ein Admin-CMS prüft, bearbeitet und erneuert die Redaktion die generierten Inhalte.

Inhaltsverzeichnis

1 Einleitung **1**

2 Umfeldanalyse **3**

2.1 Nachrichtenkonsum auf mobilen Geräten 3

2.2 Bestehende Short-Form-Angebote 4

2.3 Automatische Zusammenfassung mit Sprachmodellen 5

2.4 Personalisierung von Nachrichtenfeeds 7

3 Anforderungen **9**

3.1 Projektauftrag und Rahmenbedingungen 9

3.2 Rollen und Anwendungsfälle 10

3.3 Funktionale Anforderungen 12

3.4 Nicht-funktionale Anforderungen 14

4 Systemüberblick **16**

4.1 Bausteine des Systems 16

4.2 Vom Artikel zur Feed-Karte 18

4.3 Auslieferung und Einbindung des Feeds 20

5 Zusammenfassung **22**

Glossar **V**

Literaturverzeichnis **VI**

Abbildungsverzeichnis **VIII**

Tabellenverzeichnis **IX**

Quellcodeverzeichnis **X**

Anhang **XI**

Einsatz generativer KI XI

1 Einleitung

Nachrichten erreichen ihr Publikum heute zu einem großen Teil nicht mehr über die Startseite einer Zeitung, sondern über soziale Plattformen, Suchmaschinen und Smartphone-Apps.[1] Der Reuters Institute Digital News Report 2025 zeigt, dass der Anteil der Befragten, die pro Woche Nachrichten in Form von Social-Media-Videos nutzen, über alle untersuchten Märkte von 52 % im Jahr 2020 auf 65 % im Jahr 2025 gestiegen ist [1]. Bei den 18- bis 24-Jährigen geben 44 % an, dass soziale Netzwerke und Videoplattformen ihre wichtigste Nachrichtenquelle sind [1]. Plattformen wie TikTok oder Instagram haben dabei ein eigenes Format geprägt: kurze Inhalte im Hochformat, die einzeln den ganzen Bildschirm füllen und mit einer Wischbewegung gewechselt werden.

Klassische Nachrichtenangebote passen nur bedingt zu diesem Nutzungsverhalten. Ein Artikel auf einer Nachrichtenseite besteht aus Überschrift, Vorspann und mehreren Absätzen Fließtext und ist für das Lesen am Stück gedacht. Um denselben Inhalt in einem Short-Form-Feed anzubieten, braucht es zusätzlich eine kurze Fassung, ein passendes Bild im Hochformat und eine Einordnung in Themen. Werden diese Inhalte von Hand erstellt, entsteht für jeden Artikel zusätzlicher Aufwand in der Redaktion. Sprachmodelle können diese Aufgabe inzwischen in einer Qualität übernehmen, die in Studien mit von Menschen geschriebenen Zusammenfassungen vergleichbar ist [2]. Gleichzeitig bleiben sie fehleranfällig und erfinden mitunter Inhalte, die nicht im Ausgangstext stehen [3].

Aus dieser Ausgangslage ist gemeinsam mit dem Projektpartner Werbeberg die vorliegende Diplomarbeit entstanden. Ziel ist ein System, das aus bestehenden Nachrichtenartikeln automatisch einen Teaser, Kernpunkte, Themen-Tags und ein Titelbild erzeugt und diese Inhalte als vertikalen Feed ausliefert. Der Feed soll nicht als eigenständige App enden, sondern als Komponente in bestehende Nuxt-Websites eingebaut werden können. Die generierten Inhalte sollen nachvollziehbar und reproduzierbar bleiben und von einer Redaktion geprüft und korrigiert werden können.

[1] Die Erstfassung dieses Kapitels wurde mit dem Sprachmodell Claude Opus 5.5 (Anthropic) erstellt. Umfang und Prompt-Verlauf sind im Anhang unter „Einsatz generativer KI“ dokumentiert.

Die Arbeit gliedert sich wie folgt. Kapitel 2 ordnet das Projekt in bestehende Angebote und in die Forschung zu automatischer Zusammenfassung und Personalisierung ein. Kapitel 3 beschreibt den Projektauftrag und die daraus abgeleiteten Anforderungen. Kapitel 4 gibt einen Überblick über die Bausteine des Systems und deren Zusammenspiel, bevor die folgenden Kapitel auf die Umsetzung im Detail eingehen.

2 Umfeldanalyse

Bevor die Anforderungen an das eigene System festgelegt werden, ordnet dieses Kapitel das Projekt in sein Umfeld ein.[2] Ausgangspunkt ist die Frage nach dem Publikum: wie Nachrichten heute auf mobilen Geräten genutzt werden und welche Angebote es für kurze Nachrichtenformate bereits gibt. Danach folgen die zwei technischen Themen, auf denen wb-newsfeed aufbaut. Das erste ist die automatische Zusammenfassung von Nachrichtentexten mit Sprachmodellen, das zweite die Personalisierung von Nachrichtenfeeds. Zu beiden Themen gibt es umfangreiche Forschung, deren Ergebnisse mehrere Entscheidungen in diesem Projekt direkt beeinflusst haben.

2.1 Nachrichtenkonsum auf mobilen Geräten

Der jährlich erscheinende Digital News Report des Reuters Institute for the Study of Journalism beruht auf einer Online-Befragung in 48 Märkten und erlaubt dadurch Vergleiche über Länder und Jahre hinweg [1]. Die Ausgabe 2025 beschreibt eine deutliche Verschiebung hin zu sozialen Plattformen und Videoinhalten. Der Anteil der Befragten, die mindestens einmal pro Woche Nachrichten als Social-Media-Video konsumieren, ist über alle Märkte von 52 % im Jahr 2020 auf 65 % im Jahr 2025 gestiegen. Unter den 18- bis 24-Jährigen nennen 44 % soziale Netzwerke und Videoplattformen als ihre wichtigste Nachrichtenquelle [1].

Für Medienhäuser bedeutet diese Entwicklung, dass ein großer Teil des jungen Publikums Nachrichten zuerst in einem Feed sieht und nicht auf der eigenen Website. Feeds auf Plattformen wie TikTok, Instagram Reels oder YouTube Shorts folgen einem gemeinsamen Muster: Ein Beitrag füllt den gesamten Bildschirm, der nächste erscheint nach einer Wischbewegung, und die Reihenfolge bestimmt ein Empfehlungsalgorithmus. Ein Beitrag hat damit nur wenige Sekunden Zeit, um zu zeigen, worum es geht.

Ein weiterer Befund betrifft die Nachrichtenvermeidung. Laut demselben Bericht geben 40 % der Befragten an, Nachrichten manchmal oder oft bewusst zu meiden, im Jahr 2017 waren es noch 29 % [1]. Der Bericht fragt außerdem, welche Angebote Nachrichten für die

[2] Die Erstfassung dieses Kapitels wurde mit dem Sprachmodell Claude Opus 5.5 (Anthropic) erstellt. Umfang und Prompt-Verlauf sind im Anhang unter „Einsatz generativer KI“ dokumentiert.

Befragten attraktiver machen würden. 27 % wünschen sich zusammengefasste Fassungen von Nachrichtenartikeln [1]. Zum ersten Mal wurde 2025 auch nach KI-Chatbots als Nachrichtenquelle gefragt. Insgesamt nutzen 7 % solche Werkzeuge wöchentlich für Nachrichten, bei den unter 25-Jährigen sind es 15 % [1].

Aus diesen Zahlen lassen sich drei Folgerungen für einen Short-Form-Feed ableiten. Erstens muss der Kern einer Nachricht ohne Öffnen des Artikels erkennbar sein, also in einer Überschrift, wenigen Sätzen und einem Bild. Zweitens muss die Darstellung für Smartphones im Hochformat ausgelegt sein, weil dort die Nutzung stattfindet. Drittens ist eine Kurzfassung kein Ersatz für den Artikel. Wer sich für ein Thema interessiert, soll ohne Umweg zum vollständigen Text kommen. Diese Folgerungen finden sich in den Anforderungen in Kapitel 3 wieder. Offen bleibt, in welcher Form Medienhäuser solche Inhalte heute bereits anbieten, worum es im nächsten Abschnitt geht.

2.2 Bestehende Short-Form-Angebote

Kurze Nachrichtenformate sind nicht neu. Die indische App Inshorts fasst Nachrichten aus verschiedenen Quellen in Texten mit höchstens 60 Wörtern zusammen und wirbt mit genau dieser Begrenzung [4]. Das Angebot zeigt, dass ein Publikum für stark verkürzte Nachrichten existiert. Es ist allerdings ein eigenständiges Produkt mit eigener Marke. Ein Medienhaus, das seine Artikel dort erscheinen lassen möchte, hat keinen Einfluss darauf, wie die Kurzfassung entsteht, wie sie aussieht und welche Daten über die Nutzung gesammelt werden.

Der zweite verbreitete Weg ist die Veröffentlichung auf sozialen Plattformen. Ein Medienhaus kann dafür eigene Kurzvideos oder Bildstrecken für Instagram und TikTok produzieren. Damit erreichen sie das Publikum dort, wo es sich ohnehin aufhält, wie der vorige Abschnitt gezeigt hat. Die Inhalte entstehen dabei aber in Handarbeit, meist zusätzlich zum eigentlichen Artikel. Außerdem liegen Reichweite, Reihenfolge im Feed und Nutzungsdaten bei der Plattform und nicht beim Medium selbst. Wer auf TikTok eine Kurzmeldung sieht, landet nicht automatisch auf der Website, von der die Meldung stammt.

Ein dritter Ansatz sind Feeds auf der eigenen Website oder in der eigenen App. Hier behält das Medium die Kontrolle über Darstellung und Daten, muss aber Kurzfassungen

Kriterium	**Aggregator-App**	**Soziale Plattform**	**Eigener Feed (manuell)**	**wb-newsfeed**
Kurzfassung	extern erstellt	von Hand	von Hand	automatisch, von der Redaktion prüfbar
Ort der Darstellung	fremde App	fremde Plattform	eigene Website	eigene Website
Reihenfolge	Anbieter	Plattform-Algorithmus	Redaktion	Personalisierung im Browser
Nutzungsdaten	beim Anbieter	bei der Plattform	beim Medium	beim Medium

Tabelle 1: Vergleich verbreiteter Ansätze für Short-Form-Nachrichten

und Bilder ebenfalls erst erstellen. Für kleinere Redaktionen ist dieser zusätzliche Aufwand pro Artikel ein wesentliches Hindernis.

Tabelle 1 stellt die drei Ansätze den Zielen von wb-newsfeed gegenüber. Die Einträge der ersten drei Spalten beschreiben die Ansätze allgemein und nicht ein bestimmtes Produkt.

Die Lücke, die das Projekt schließen soll, liegt in der Kombination aus der letzten Spalte. Bestehende Artikel sollen ohne zusätzliche Handarbeit zu Feed-Inhalten werden, und der Feed soll als Teil der bestehenden Website laufen. Damit das gelingt, muss die Kurzfassung automatisch entstehen. Wie gut Sprachmodelle diese Aufgabe erfüllen und wo ihre Grenzen liegen, behandelt der folgende Abschnitt.

2.3 Automatische Zusammenfassung mit Sprachmodellen

In der Forschung zur automatischen Textzusammenfassung werden zwei Verfahren unterschieden. Extraktive Verfahren wählen Sätze aus dem Originaltext aus und setzen sie zusammen. Abstraktive Verfahren formulieren eine neue, kürzere Fassung in eigenen Worten [3]. Für einen Teaser in einem Feed ist nur der zweite Weg sinnvoll, weil die ersten Sätze eines Nachrichtenartikels selten eine in sich geschlossene Kurzfassung ergeben.

Mit großen Sprachmodellen (Large Language Model (LLM)) hat sich die abstraktive Zusammenfassung stark verändert. Goyal, Li und Durrett verglichen Zusammenfassungen

von GPT-3, die ohne Training auf die Aufgabe nur durch eine Anweisung im Prompt entstanden, mit Modellen, die speziell für Nachrichtenzusammenfassung trainiert worden waren [5]. Die befragten Personen bevorzugten in ihrer Studie deutlich die Texte von GPT-3. Gleichzeitig zeigten die Autor:innen, dass gängige automatische Metriken diesen Unterschied nicht abbilden [5]. Zhang et al. untersuchten zehn Sprachmodelle in einer großen Studie mit menschlicher Bewertung [2]. Entscheidend für die Qualität war dabei nicht die Größe des Modells, sondern ob es auf das Befolgen von Anweisungen trainiert wurde. Die Zusammenfassungen dieser Modelle wurden ähnlich gut bewertet wie Texte, die freiberufliche Autor:innen für die Studie geschrieben hatten [2]. Für dieses Projekt ist der erste Befund besonders wichtig. Er spricht dafür, dass auch ein mittelgroßes Modell, das lokal auf einer einzelnen Grafikkarte läuft, für Teaser ausreicht, solange es auf Anweisungen trainiert ist.

Das größte Problem abstraktiver Zusammenfassungen sind sogenannte Halluzinationen. Maynez et al. zeigten an Nachrichtenzusammenfassungen, dass Modelle regelmäßig Inhalte erzeugen, die nicht durch den Ausgangstext gedeckt sind [3]. Ji et al. unterscheiden in ihrer Übersichtsarbeit zwei Arten [6]. Intrinsische Halluzinationen widersprechen dem Ausgangstext, etwa durch eine vertauschte Zahl. Extrinsische Halluzinationen fügen Informationen hinzu, die sich aus dem Ausgangstext weder bestätigen noch widerlegen lassen. Im Nachrichtenbereich sind beide Arten heikel, weil ein Teaser unter dem Namen der Redaktion erscheint.

Aus diesen Ergebnissen leiten sich mehrere Entscheidungen für wb-newsfeed ab, die in den folgenden Kapiteln genauer beschrieben werden. Das Modell bekommt eine eng begrenzte Aufgabe mit festem Ausgabeformat: höchstens drei Sätze Teaser, drei Kernpunkte und Tags ausschließlich aus einer vorgegebenen Liste. Titel und Kategorie, die die Redaktion bereits gesetzt hat, darf das Modell nicht überschreiben. Jedes Ergebnis wird mit Modell und Parametern versioniert, damit sich nachvollziehen lässt, wie ein Text entstanden ist. Die Redaktion kann jeden generierten Text im Admin-Content-Management-System (CMS) prüfen und von Hand korrigieren. Eingesetzt wird ein Modell der Qwen3-Familie, das sowohl einen Modus mit vorgeschaltetem Denkprozess als auch einen Modus für direkte Antworten bietet [7]. Für Teaser genügt der direkte Modus, für die anspruchsvollere Planung von Bildmotiven wird der Denkmodus genutzt.

Die Kurzfassung allein macht aber noch keinen Feed. Ebenso wichtig ist, in welcher Reihenfolge die Beiträge erscheinen, was Thema des nächsten Abschnitts ist.

2.4 Personalisierung von Nachrichtenfeeds

Empfehlungssysteme für Nachrichten unterscheiden sich in einigen Punkten von Systemen für Filme oder Produkte. Raza und Ding nennen in ihrer Übersichtsarbeit unter anderem die kurze Lebensdauer von Nachrichten, das Fehlen von Informationen über neue Nutzer:innen und neue Artikel sowie den Umstand, dass Interessen meist nur indirekt aus dem Verhalten ablesbar sind [8]. Ein Artikel über ein Fußballspiel ist am nächsten Tag kaum noch relevant, während ein Film jahrelang empfohlen werden kann.

Wu et al. veröffentlichten mit MIND einen großen Datensatz für Nachrichtenempfehlung, der aus den Klickdaten von Microsoft News entstanden ist und rund eine Million Nutzer:innen sowie über 160 000 englischsprachige Artikel umfasst [9]. Ihre Vergleiche bestehender Verfahren zeigen, dass die Qualität der Empfehlungen stark davon abhängt, wie gut ein System den Inhalt der Artikel versteht und wie gut es die Interessen der Nutzer:innen modelliert [9]. Inhaltsbasierte Verfahren, die Artikel über ihren Text vergleichen, haben hier einen Vorteil. Sie funktionieren auch für Artikel, die gerade erst veröffentlicht wurden und noch keine Klicks haben.

Eine verbreitete Technik für den inhaltlichen Vergleich sind Embeddings. Ein Embedding-Modell bildet einen Text auf einen Vektor ab, und die Ähnlichkeit zweier Texte lässt sich dann über den Winkel zwischen ihren Vektoren bestimmen. wb-newsfeed verwendet dafür das Modell BGE-M3, das über 100 Sprachen unterstützt und Texte mit bis zu 8 192 Tokens verarbeiten kann [10]. Für deutschsprachige Nachrichtenartikel ist die Mehrsprachigkeit eine Voraussetzung, die rein englischsprachige Modelle nicht erfüllen.

Personalisierung hat auch eine Kehrseite. Pariser beschreibt unter dem Begriff Filterblase die Gefahr, dass Nutzer:innen durch personalisierte Auswahl nur noch Inhalte sehen, die ihre bisherigen Interessen bestätigen [11]. Ein bekanntes Gegenmittel aus dem Information Retrieval ist die Maximal Marginal Relevance von Carbonell und Goldstein [12]. Dabei wird bei der Sortierung nicht nur die Relevanz eines Dokuments berücksichtigt, sondern auch, wie sehr es sich von den bereits gewählten Dokumenten unterscheidet.

Für wb-newsfeed ergeben sich daraus zwei Leitlinien. Die Personalisierung soll ohne Benutzerkonto funktionieren, weil ein eingebetteter Feed auf einer fremden Website keine Anmeldung voraussetzen kann. Das Interessenprofil bleibt deshalb im Browser

und wird nur aus dem Verhalten im Feed gelernt, etwa aus Verweildauer, Likes und gelesenen Artikeln. Außerdem soll der Feed bewusst Abwechslung erzeugen. Er begrenzt daher, wie viele Artikel desselben Themas hintereinander erscheinen, und reserviert regelmäßig Plätze für Themen außerhalb der bisherigen Interessen. Wie diese Leitlinien in konkrete Anforderungen übersetzt wurden, beschreibt das nächste Kapitel.

3 Anforderungen

Die Anforderungen an wb-newsfeed stammen aus zwei Dokumenten.[3] Das erste ist das Briefing des Projektpartners Werbeberg vom 30. Jänner 2026, das den Funktionsumfang und die technischen Rahmenbedingungen festlegt. Das zweite ist der Diplomarbeitsantrag vom 19. Februar 2026, in dem die geplanten Ergebnisse auf die beiden Autoren aufgeteilt sind. Dieses Kapitel fasst beide Quellen zusammen, beschreibt die Personen, die mit dem System arbeiten, und ordnet die Anforderungen in funktionale und nichtfunktionale Anforderungen. Jede Anforderung trägt eine Kennung, über die in späteren Kapiteln auf sie verwiesen wird.

3.1 Projektauftrag und Rahmenbedingungen

<[**Ein bis zwei Sätze zum Projektpartner Werbeberg ergänzen: Tätigkeitsfeld, Ansprechperson, warum das Thema für das Unternehmen interessant ist.**]> <==

Der Auftrag lässt sich in einem Satz zusammenfassen: Aus gewöhnlichen Nachrichtenartikeln sollen automatisch Short-Form-Inhalte entstehen, die in einem vertikal scrollbaren Feed angezeigt werden, und dieser Feed soll sich als Komponente in bestehende Nuxt-Projekte einbauen lassen. Das Briefing beschreibt damit nicht nur eine Anwendung, sondern einen Baustein, den eine Website übernehmen kann, ohne ihr bestehendes CMS zu ersetzen.

Daraus ergeben sich drei Teilaufgaben. Die erste ist die Generierung: Aus Titel, Text und Kategorie eines Artikels entstehen ein Teaser, Kernpunkte und weitere Metadaten, die gespeichert werden. Die zweite ist die Bereitstellung über eine Representational State Transfer (REST)-Schnittstelle, die neue und geänderte Inhalte ausliefert. Die dritte ist die Darstellung im Feed selbst, die als wiederverwendbares Package gebaut wird. Möchte jemand einen Artikel vollständig lesen, übergibt der Feed lediglich eine eindeutige Kennung an die einbindende Website. Wie diese den Artikel anzeigt, bleibt ihr überlassen.

[3] Die Erstfassung dieses Kapitels wurde mit dem Sprachmodell Claude Opus 5.5 (Anthropic) erstellt. Umfang und Prompt-Verlauf sind im Anhang unter „Einsatz generativer KI“ dokumentiert.

ID	Rahmenbedingung
T1	Oberfläche mit Nuxt 4 (SSR) und TypeScript
T2	Gestaltung mit Tailwind CSS in Version 4
T3	Einheitliche Code-Prüfung mit ESLint im gesamten Projekt
T4	Feed-Oberfläche als wiederverwendbares Package
T5	Backend für asynchrone Generierung, Verwaltung und Bereitstellung der Inhalte

Tabelle 2: Technische Rahmenbedingungen laut Briefing

Das Briefing gibt außerdem den technischen Rahmen vor, der in Tabelle 2 zusammengefasst ist. Für die Oberfläche sind Nuxt 4 mit Server-Side Rendering (SSR), TypeScript und Tailwind CSS in Version 4 vorgegeben. Für das Backend schreibt das Briefing keine Sprache vor. Verlangt wird nur, dass es die Generierung asynchron verarbeitet und die Inhalte verwaltet und bereitstellt.

Die Arbeit ist entlang dieser Teilaufgaben aufgeteilt. David Windischbauer verantwortet das Backend mit Schnittstelle, Datenbank, Warteschlange, Text- und Bildgenerierung sowie die Dokumentation der Schnittstellen. Stefan Schachner verantwortet die Feed-App, das Admin-CMS, das Komponenten-Package und dessen Einbindung in fremde Websites. Die Entwicklung fand zwischen Juli und September 2026 statt. Das gemeinsame Git-Repository umfasst in diesem Zeitraum 258 Commits.

Zur Abgrenzung wurden zu Projektbeginn einige Punkte festgelegt. Das System ist kein Ersatz für ein Redaktionssystem, Artikel entstehen weiterhin in der Redaktion und gelangen über eine Schnittstelle oder einen Import in wb-newsfeed. Benutzerkonten für das Publikum sind nicht vorgesehen. Mehrere Erweiterungsideen, etwa eine Audiofassung oder Teaser-Varianten für unterschiedliche Zielgruppen, sind als optional markiert. Sie werden in Abschnitt 3.3 getrennt von den Pflichtanforderungen geführt. Wer mit dem System im Alltag arbeitet, beschreibt der nächste Abschnitt.

3.2 Rollen und Anwendungsfälle

Mit wb-newsfeed arbeiten drei Gruppen von Personen, deren Erwartungen sich deutlich unterscheiden.

Leser:innen sehen den Feed auf dem Smartphone, meist eingebettet in die Website eines Mediums. Sie haben kein Konto und erwarten, dass der Feed sofort lädt und sich wie gewohnte Short-Form-Plattformen bedienen lässt.

Rolle	User Story
Leser:in	Als Leser:in möchte ich durch kurze Zusammenfassungen wischen, um in kurzer Zeit einen Überblick über die aktuellen Nachrichten zu bekommen.
Leser:in	Als Leser:in möchte ich einen Artikel mit einem Tippen vollständig öffnen, um mehr über ein Thema zu erfahren.
Leser:in	Als Leser:in möchte ich den Feed nach Kategorien filtern, um nur Themen zu sehen, die mich gerade interessieren.
Leser:in	Als Leser:in möchte ich, dass der Feed mit der Zeit mehr Artikel zeigt, die zu meinen Interessen passen, ohne dass ich mich anmelden muss.
Leser:in	Als Leser:in möchte ich Artikel liken, kommentieren und teilen können.
Redakteur:in	Als Redakteur:in möchte ich einen Artikel anlegen und automatisch Teaser, Kernpunkte und Tags erhalten, um keine Kurzfassung selbst schreiben zu müssen.
Redakteur:in	Als Redakteur:in möchte ich generierte Texte korrigieren und frühere Fassungen wiederherstellen können.
Redakteur:in	Als Redakteur:in möchte ich Artikel aus einem bestehenden RSS-Feed übernehmen, um sie nicht abtippen zu müssen.
Redakteur:in	Als Redakteur:in möchte ich sehen, wie oft ein Artikel angesehen und wie lange er gelesen wurde.
Entwickler:in	Als Entwickler:in möchte ich den Feed mit einer einzigen Komponente in mein Nuxt-Projekt einbauen.
Entwickler:in	Als Entwickler:in möchte ich Farben und Schriften des Feeds an meine Website anpassen.
Entwickler:in	Als Entwickler:in möchte ich selbst entscheiden, was beim Öffnen eines Artikels passiert.

Tabelle 3: Auswahl der User Stories je Rolle

Redakteur:innen arbeiten im Admin-CMS. Sie legen Artikel an oder importieren sie, prüfen die generierten Inhalte, korrigieren sie bei Bedarf und veröffentlichen die Artikel.

Entwickler:innen der Host-Website binden den Feed in ihre bestehende Nuxt-Website ein. Für sie zählt eine überschaubare Einrichtung, eine klare Dokumentation und die Möglichkeit, das Aussehen an die eigene Marke anzupassen.

Die Anforderungen wurden für jede dieser Gruppen als User Stories formuliert. Tabelle 3 zeigt eine Auswahl, die den Kern des Systems abdeckt.

Aus den User Stories ergeben sich zwei zentrale Anwendungsfälle, die das ganze System durchlaufen. Im ersten legt ein:e Redakteur:in einen Artikel an. Das System übernimmt Titel, Text und Kategorie, erzeugt im Hintergrund Teaser, Kernpunkte und Tags und zeigt den Fortschritt im Admin-CMS an. Nach der Prüfung wird der Artikel veröffentlicht

ID	Anforderung
F1	Vertikal scrollbarer Short-Form-Feed mit einer Karte pro Artikel
F2	Anzeige von KI-generierten Kurzinhalten (Teaser) auf den Karten
F3	Übergabe einer eindeutigen Kennung an die Host-Website, wenn ein Artikel geöffnet wird
G1	Generierung der Kurzinhalte aus strukturierten Artikeldaten
G2	Dauerhafte Speicherung der generierten Inhalte
E1	Einbindung des Feeds als Nuxt-Komponente in bestehende Nuxt-Projekte
E2	Erkennung neuer und geänderter Inhalte im Feed
E3	Bereitstellung der Inhalte über eine REST-Schnittstelle
D1	Reproduzierbare und konfigurierbare KI-Generierung
D2	Asynchrone Verarbeitung neuer und geänderter Inhalte
D3	Versionierung der generierten Inhalte
D4	Unabhängigkeit von einem bestimmten CMS

Tabelle 4: Funktionale Pflichtanforderungen

und erscheint beim nächsten Abruf im Feed. Ändert sich später der Artikeltext, wird der Teaser neu erzeugt, während eine von Hand geänderte Fassung erhalten bleibt.

Im zweiten Anwendungsfall öffnet eine Person die Website eines Mediums auf dem Smartphone. Der Feed lädt die ersten Artikel, zeigt die erste Karte und lädt weitere Karten nach, während gewischt wird. Aus dem Verhalten, etwa wie lange eine Karte betrachtet oder ob ein Artikel geöffnet wird, lernt der Feed Interessen und ordnet die folgenden Karten neu. Tippt die Person auf „Weiterlesen“, meldet der Feed die Kennung des Artikels an die Website, die den vollständigen Text anzeigt.

Beide Anwendungsfälle zeigen, dass Generierung und Darstellung nur über die Schnittstelle miteinander verbunden sind. Die genauen Anforderungen an beide Seiten folgen im nächsten Abschnitt.

3.3 Funktionale Anforderungen

Tabelle 4 listet die funktionalen Pflichtanforderungen. Die Kennungen mit F, G und E stammen aus dem Briefing, jene mit D aus dem Diplomarbeitsantrag. Anforderungen, die in beiden Dokumenten vorkommen, sind nur einmal angeführt.

Einige dieser Anforderungen sind im Briefing bewusst knapp formuliert und mussten für die Umsetzung präzisiert werden.

F3 legt fest, dass der Feed beim Öffnen eines Artikels nur eine Kennung übergibt. Der Feed zeigt also nicht selbst eine eigene Artikelseite an, sondern meldet über ein Ereignis die Artikel-ID an die einbindende Website. Damit bleibt die Website für Navigation, Layout und gegebenenfalls Bezahlschranken zuständig.

G1 wurde so präzisiert, dass aus Titel, Text und Kategorie eines Artikels ein Teaser mit höchstens drei Sätzen, drei Kernpunkte und ein bis zwei Tags entstehen. Die Tags stammen aus einer festen Liste von Unterkategorien je Hauptkategorie. Freie Schlagwörter wie Personen- oder Firmennamen würden zu sehr vielen Tags mit jeweils nur einem Artikel führen und wären für die Filterung im Feed unbrauchbar. Als Hauptkategorien sind Politik, Wirtschaft, Sport, Technologie, Kultur und Chronik festgelegt. Zusätzlich zum Text entsteht auf Wunsch ein Titelbild im Hochformat.

E2 verlangt, dass der Feed neue oder geänderte Inhalte erkennt, ohne dass die Seite neu geladen werden muss. Das Briefing lässt offen, ob das über regelmäßige Abfragen oder über eine Push-Verbindung geschieht.

D1 verlangt, dass derselbe Artikel mit denselben Einstellungen dasselbe Ergebnis liefert. Bei Sprachmodellen ist das nicht selbstverständlich, weil die Ausgabe standardmäßig zufällig gezogen wird. Modell, Temperatur und Seed müssen deshalb einstellbar sein und bei jedem Ergebnis mitgespeichert werden.

D3 geht über das bloße Speichern hinaus. Jede generierte oder von Hand geänderte Fassung soll als eigene, unveränderliche Version erhalten bleiben und wiederhergestellt werden können. Das ist die Grundlage dafür, dass Redakteur:innen Ergebnisse vergleichen und die in Abschnitt 2.3 beschriebenen Fehler eines Sprachmodells nachvollziehen können.

D4 bedeutet, dass Artikel aus beliebigen Quellen ins System gelangen können. Neben der direkten Anlage über die Schnittstelle ist dafür ein Import aus RSS- und Atom-Feeds vorgesehen, sodass bestehende Nachrichtenquellen ohne eigene Anpassung angebunden werden können.

Neben den Pflichtanforderungen nennt das Briefing mehrere Erweiterungsideen, die in Tabelle 5 zusammengefasst sind. Sie fließen nicht in die Bewertung der Pflichtanforderungen ein, wurden aber bei der Architektur von Anfang an berücksichtigt.

Die funktionalen Anforderungen beschreiben, was das System leisten soll. Wie schnell, wie sicher und wie gut wartbar es dabei sein muss, legen die nicht-funktionalen Anforderungen im folgenden Abschnitt fest.

ID	Optionale Erweiterung
O1	Personalisierung der Reihenfolge im Feed
O2	Personalisierung der Inhalte, etwa Teaser-Varianten je Zielgruppe
O3	Admin-Dashboard auf Basis von Nuxt 4
O4	Weitergabe von Nutzungsdaten über Client-Ereignisse oder Webhooks
O5	Audiofassung der Inhalte

Tabelle 5: Optionale Erweiterungen laut Briefing

ID	Anforderung
S1	Responsive Oberfläche, ausgelegt für Smartphones im Hochformat
S2	Performantes Rendering und schrittweises Nachladen der Inhalte
S3	Konfigurierbare Darstellung und Gestaltung des Feeds
S4	Ereignisse für Tracking und Analytics
S5	Technische Dokumentation zur Einbindung
D5	Performance des Backends durch Caching und Warteschlangen
D6	Dokumentation von Schnittstellen und Architektur

Tabelle 6: Nicht-funktionale Anforderungen

3.4 Nicht-funktionale Anforderungen

Neben den technischen Rahmenbedingungen aus Tabelle 2 enthält der Diplomarbeitsantrag mehrere Ziele, die die Qualität des Systems betreffen. Sie sind in Tabelle 6 zusammengefasst und werden anschließend erläutert.

Benutzbarkeit auf dem Smartphone (S1). Der Feed muss auf Bildschirmen ab einer Breite von 375 Pixeln vollständig bedienbar sein. Jede Karte füllt die Höhe des Bildschirms aus und rastet beim Wischen ein, sodass immer genau ein Artikel sichtbar ist. Bedienelemente wie Like oder Teilen müssen mit dem Daumen erreichbar sein.

Ladezeit und Speicherbedarf (S2, D5). Ein Feed kann sehr viele Artikel enthalten, und jede Karte trägt ein großes Bild. Würden beim Start alle Artikel geladen und alle Karten gezeichnet, stiegen Datenmenge und Speicherbedarf mit jedem neuen Artikel. Der Feed soll daher Artikel in Seiten laden und nur die Karten rendern, die sich in der Nähe der sichtbaren Karte befinden. Auf der Seite des Backends darf eine langsame Generierung keine Anfrage an den Feed blockieren. Generierung und Auslieferung müssen deshalb getrennt sein, und häufig abgefragte Daten sollen zwischengespeichert werden. Da der Feed regelmäßig auf Änderungen prüft (E2), soll eine unveränderte Antwort ohne erneute Übertragung des Inhalts beantwortet werden können.

Anpassbarkeit (S3). Websites, die den Feed einbinden, haben eigene Farben und Schriften. Das Aussehen des Feeds muss daher ohne Änderung am Code des Packages anpassbar sein. Einzelne Bereiche einer Karte sollen sich zusätzlich durch eigene Inhalte ersetzen lassen.

Messbarkeit (S4). Für Redaktionen ist es wichtig zu wissen, welche Artikel im Feed funktionieren. Der Feed muss deshalb Ereignisse wie das erste Sichtbarwerden einer Karte, das Öffnen eines Artikels oder ein Like nach außen melden. Die einbindende Website kann diese Ereignisse an ein eigenes Analysewerkzeug weitergeben.

Dokumentation (S5, D6). Die Schnittstelle soll vollständig und maschinenlesbar beschrieben sein, sodass andere Entwickler:innen sie ohne Blick in den Quellcode nutzen können. Für das Package ist eine Anleitung nötig, die alle Schritte der Einbindung, die Konfiguration und alle Ereignisse beschreibt.

Sicherheit und Datenschutz. Diese Punkte sind im Briefing nicht ausdrücklich genannt, ergeben sich aber aus dem Einsatz auf öffentlichen Websites. Schreibende Zugriffe auf die Schnittstelle, etwa das Anlegen oder Löschen von Artikeln, dürfen nur mit einem Schlüssel möglich sein. Der Feed selbst muss ohne Anmeldung funktionieren. Das Interessenprofil für die Personalisierung soll das Gerät der Leser:innen nicht verlassen, an den Server gehen nur anonyme Ereignisse für die Statistik. Eine weitere Vorgabe des Projektteams betrifft die Kosten: Text- und Bildgenerierung sollen lokal auf eigener Hardware laufen können, damit keine laufenden Kosten pro Artikel bei einem externen Anbieter entstehen.

Mit diesen Anforderungen steht fest, was das System leisten muss. Wie die einzelnen Bausteine aufgebaut sind, um sie zu erfüllen, zeigt das folgende Kapitel.

4 Systemüberblick

Dieses Kapitel beschreibt, aus welchen Bausteinen wb-newsfeed besteht und wie sie zusammenarbeiten.[4] Es dient als Landkarte für die folgenden Kapitel, in denen die einzelnen Teile genauer behandelt werden. Zuerst werden die Komponenten und ihre Verbindungen vorgestellt. Danach folgt der Weg eines Artikels von der Anlage bis zur fertigen Karte im Feed und zuletzt die Auslieferung des Feeds an eine einbindende Website.

4.1 Bausteine des Systems

Abbildung 1 zeigt die Bausteine des Systems und ihre Verbindungen. Im oberen Bereich liegen die Oberflächen, über die Menschen mit dem System arbeiten. In der Mitte steht die zentrale Schnittstelle, darunter die Dienste für Datenhaltung und Generierung.

Admin-CMS. Über das Admin-CMS legen Redakteur:innen Artikel an, importieren RSS-Feeds, prüfen generierte Inhalte und ändern die Einstellungen für die Modelle. Es ist eine eigene Nuxt-Anwendung. Anfragen an die Schnittstelle laufen nicht direkt aus dem Browser, sondern über eine Server-Route der Nuxt-Anwendung. Diese ergänzt den geheimen Schlüssel für schreibende Zugriffe, sodass er nie im Browser landet.

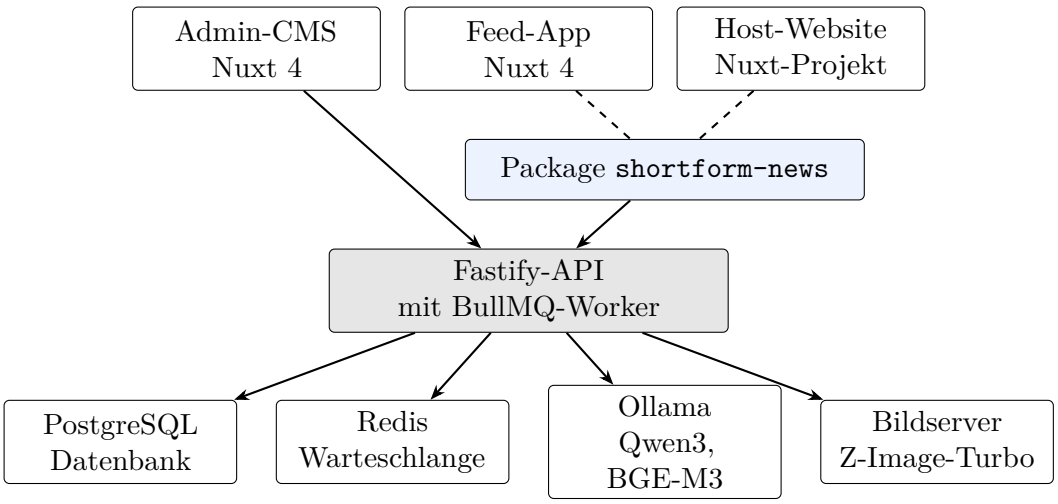


Abbildung 1: Bausteine von wb-newsfeed und ihre Verbindungen

[4] Die Erstfassung dieses Kapitels wurde mit dem Sprachmodell Claude Opus 5.5 (Anthropic) erstellt. Umfang und Prompt-Verlauf sind im Anhang unter „Einsatz generativer KI“ dokumentiert.

Feed-App und Package. Der eigentliche Feed ist im Package `@wb-news/shortform-news` umgesetzt. Die Feed-App des Projekts ist selbst nur eine dünne Hülle um dieses Package und verhält sich damit genauso wie jede fremde Website, die den Feed einbindet. Dadurch wird jede Änderung am Package automatisch in derselben Form getestet, in der sie später bei einem Host ankommt.

Schnittstelle und Worker. Die Schnittstelle ist mit dem Node.js-Framework Fastify umgesetzt und in einer OpenAPI-3.1-Spezifikation mit 35 Operationen beschrieben. Lesende Endpunkte, die der Feed ohne Anmeldung braucht, sind öffentlich. Alle anderen verlangen einen Schlüssel im Header `x-api-key`, ohne konfigurierten Schlüssel startet die Schnittstelle gar nicht. Im selben Prozess läuft ein Worker der Bibliothek BullMQ, der Aufträge aus der Warteschlange abarbeitet. Die Trennung von Schnittstelle und Worker erfolgt damit nicht über getrennte Prozesse, sondern über die Warteschlange. Ein Worker kann bei Bedarf trotzdem als eigener Prozess ausgelagert werden, weil beide Seiten nur über Redis miteinander sprechen.

Datenhaltung. Artikel, Aufträge, Versionen, Tags, Kommentare, Nutzungsereignisse und Einstellungen liegen in einer PostgreSQL-Datenbank. Der Zugriff erfolgt über das Object-Relational Mapping (ORM) Drizzle, in dem das Schema als TypeScript-Code definiert ist. Redis dient ausschließlich als Speicher für die Warteschlange.

Generierung. Für Texte und Embeddings läuft Ollama als lokaler Dienst. Standardmäßig werden das Sprachmodell `qwen3:14b` [7] und das Embedding-Modell `bge-m3` [10] verwendet. Titelbilder erzeugt ein eigener Bildserver in Python. Er verwendet standardmäßig das Diffusionsmodell Z-Image-Turbo mit sechs Milliarden Parametern [13], alternativ FLUX.1-schnell. Der Bildserver bietet denselben Endpunkt wie die Bild-Application Programming Interface (API) von OpenAI an, sodass er sich bei Bedarf gegen einen anderen kompatiblen Dienst tauschen lässt. Optional kann statt des lokalen Servers die kostenpflichtige Bildgenerierung von Google Gemini verwendet werden.

Der gesamte Code liegt in einem Monorepo, das mit pnpm-Workspaces verwaltet wird. Die drei Anwendungen liegen unter `apps/`, das Package unter `packages/` und der Bildserver unter `services/`. PostgreSQL, Redis und Ollama starten über Docker Compose. Der Bildserver läuft dagegen direkt auf dem Entwicklungsrechner, damit er ohne zusätzliche Schicht auf die Graphics Processing Unit (GPU) zugreifen kann. Entwickelt und getestet wurde auf einer Grafikkarte mit 16 GB Video Random Access Memory (VRAM). Damit Text- und Bildmodell auf dieser Karte Platz finden, gibt

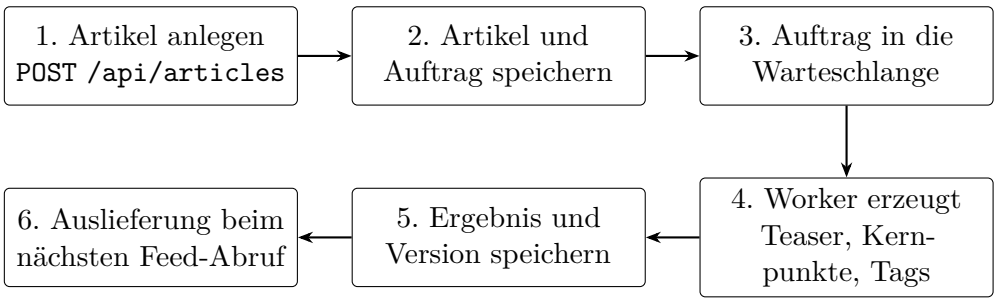


Abbildung 2: Ablauf der asynchronen Generierung eines Artikels

Ollama den Speicher nach jeder Anfrage für ein Bildmotiv sofort wieder frei, und der Bildserver lädt, sofern genug Speicher frei ist, jeweils nur die gerade rechnende Komponente des Modells auf die Grafikkarte.

Wie diese Bausteine bei der Verarbeitung eines einzelnen Artikels zusammenspielen, zeigt der nächste Abschnitt.

4.2 Vom Artikel zur Feed-Karte

Die Generierung von Inhalten dauert je nach Modell und Auslastung zwischen einigen Sekunden und über einer Minute. <[**Mit eigenen Messwerten belegen, z. B. aus** `jobs.processing_time_ms.`]> <== Würde die Schnittstelle so lange auf das Ergebnis warten, wäre das Admin-CMS blockiert, und bei einem Import vieler Artikel würden Anfragen in eine Zeitüberschreitung laufen. Die Generierung läuft deshalb asynchron, wie Abbildung 2 zeigt.

Legt ein:e Redakteur:in einen Artikel an, speichert die Schnittstelle ihn sofort in der Datenbank. Bis das Sprachmodell geantwortet hat, erhält der Artikel vorläufige Werte aus einer einfachen Heuristik, die anhand von Schlüsselwörtern eine Kategorie bestimmt und den Textanfang als Teaser verwendet. Gleichzeitig entsteht ein Eintrag in der Tabelle der Aufträge mit dem Status `pending`, und der Auftrag wird in die Warteschlange gestellt. Die Antwort an das Admin-CMS enthält die Kennung des Artikels und die Kennung des Auftrags, über die sich der Fortschritt abfragen lässt. Dieser Ablauf ist für alle Wege gleich, über die Artikel ins System kommen, also für das Admin-CMS, für direkte Aufrufe der Schnittstelle und für den RSS-Import.

Der Worker holt den Auftrag aus der Warteschlange, setzt den Status auf `processing` und schickt den Artikeltext mit einer festen Anweisung an Ollama. Listing 1 zeigt den Aufruf. Das Modell muss im JavaScript Object Notation (JSON)-Format antworten,

der Denkmodus ist für Teaser abgeschaltet, und Temperatur und Seed stammen aus den Einstellungen. Standardmäßig sind das eine Temperatur von 0 und der Seed 42, womit gleiche Eingaben zu gleichen Ausgaben führen (Anforderung D1).

Listing 1: Aufruf des Sprachmodells über Ollama (gekürzt aus `summarizer.ts`)

```
const response = await fetch(`${ollamaUrl}/api/generate`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    model: aiModel,              // Standard: qwen3:14b
    prompt: promptText,
    stream: false,
    format: 'json',              // Antwort als JSON-Objekt
    think: false,                // kein Denkmodus fuer Teaser
    options: generationOptions   // { temperature, seed }
  }),
  signal: controller.signal      // Abbruch nach Zeitlimit
});
```

Das Ergebnis schreibt der Worker in den Artikel und legt zusätzlich eine neue Version in der Tabelle `generation_versions` an. Eine Version enthält Teaser, Kernpunkte, Titel, Kategorie und Tags sowie das verwendete Modell, Temperatur, Seed und einen SHA-256-Hash des Artikeltexts. Über den Hash lässt sich später feststellen, auf welchem Stand des Textes eine Fassung beruht. Versionen werden nie verändert. Wird eine ältere Fassung wiederhergestellt, entsteht dafür eine neue Version, die auf die alte verweist (Anforderung D3).

Für den Fall, dass ein Schritt fehlschlägt, gibt es mehrere Rückfallebenen. Ist Ollama nicht erreichbar, liefert die Heuristik ein Ergebnis, das in Auftrag und Version ausdrücklich als Fallback markiert wird. Schlägt ein Auftrag aus anderen Gründen fehl, wiederholt BullMQ ihn bis zu dreimal mit wachsendem Abstand. Scheitert nur das Titelbild, bleibt der Text trotzdem gespeichert, und der Auftrag vermerkt den Fehler beim Bild.

Ein Titelbild entsteht standardmäßig nur auf Anforderung im Admin-CMS oder automatisch, wenn diese Option in den Einstellungen aktiviert ist. In diesem Fall plant das Sprachmodell zuerst das Bildmotiv in zwei Schritten, einem Entwurf und einer Prüfung gegen die Schlagzeile. Danach rendert der Bildserver ein Bild mit 768×1344 Pixeln, was dem Seitenverhältnis 9:16 einer Feed-Karte entspricht. Die Gründe für dieses zweistufige Vorgehen und die Wahl des Bildmodells werden im Kapitel zur Bildgenerierung erläutert.

Ändert ein:e Redakteur:in später den Text eines Artikels, vergleicht die Schnittstelle den neuen mit dem gespeicherten Stand und stellt bei Bedarf automatisch einen neuen Auftrag ein (Anforderung D2). Werden dagegen Teaser oder Kernpunkte von Hand

geändert, entsteht eine Version mit der Quelle `manual`, und es wird nichts neu generiert. Ein Auftrag, der bereits in der Warteschlange wartet, überschreibt eine solche Handarbeit ebenfalls nicht.

Am Ende dieses Ablaufs liegt ein veröffentlichter Artikel mit allen Inhalten für eine Feed-Karte in der Datenbank. Wie er von dort in den Feed auf einer fremden Website gelangt, beschreibt der folgende Abschnitt.

4.3 Auslieferung und Einbindung des Feeds

Der Feed bezieht alle Artikel über den öffentlichen Endpunkt `GET /api/feed`. Dieser liefert veröffentlichte Artikel samt Tags, Zählern für Likes, Kommentare und Teilungen sowie einer Liste inhaltlich ähnlicher Artikel. Die Liste der ähnlichen Artikel berechnet die Schnittstelle aus den Embeddings und stellt sie der Personalisierung im Browser zur Verfügung. Die Artikel kommen seitenweise, gesteuert über die Parameter `limit` und `offset`.

Weil der Feed regelmäßig auf Änderungen prüft, standardmäßig alle 60 Sekunden, kommen viele Anfragen, deren Antwort sich nicht geändert hat. Die Schnittstelle hält die Liste der veröffentlichten Artikel deshalb im Arbeitsspeicher und prüft pro Anfrage nur mit einer kleinen Abfrage, ob sich etwas geändert hat. Zusätzlich trägt jede Antwort ein `ETag`, also eine Kennung ihres Inhalts. Schickt der Browser diese Kennung bei der nächsten Anfrage mit und hat sich nichts geändert, antwortet die Schnittstelle mit dem Statuscode `304 Not Modified` ohne Inhalt, wie es die HTTP-Spezifikation vorsieht [14]. Damit erfüllt die Auslieferung die Anforderung E2 mit regelmäßigen Abfragen, ohne bei jeder Abfrage die gesamte Liste zu übertragen.

Auf der Seite des Browsers übernimmt die Komponente `ShortformNewsFeed` aus dem Package alle Aufgaben des Feeds. Sie lädt die erste Seite mit zehn Artikeln und holt weitere, sobald nur noch drei Karten unter der sichtbaren liegen. Gerendert werden nur die sichtbare Karte und je zwei davor und danach, alle anderen Karten sind leere Platzhalter mit derselben Höhe. Bilder werden dadurch erst geladen, wenn ihre Karte in die Nähe kommt. Ebenfalls im Browser läuft die Personalisierung: Das Interessenprofil wird im `localStorage` gespeichert, und nur anonyme Ereignisse wie Verweildauer oder Lesedauer gehen an die Schnittstelle.

Für einen Host besteht die Einbindung im Kern aus wenigen Schritten. Das Package wird installiert und als Nuxt-Modul registriert, das die Komponenten global zur Verfügung stellt. Weil die Komponenten mit Tailwind CSS gestaltet sind, muss der Host Tailwind die Dateien des Packages bekannt machen. Die Adresse des Hosts muss außerdem in der Konfiguration der Schnittstelle freigeschaltet sein, da der Browser Anfragen an eine fremde Adresse sonst über Cross-Origin Resource Sharing (CORS) blockiert. Listing 2 zeigt, wie der Feed danach in einer Seite des Hosts verwendet wird.

Listing 2: Einbindung des Feeds in eine Seite der Host-Website

```
<template>
  <ShortformNewsFeed
    api-url="https://api.example.at"
    @article-read="openArticle" />
</template>

<script setup lang="ts">
// Der Feed liefert nur die Artikel-ID (Anforderung F3),
// der Host entscheidet selbst, was passiert.
const openArticle = (id: number) => navigateTo(`/artikel/${id}`);
</script>
```

Der Host muss nur auf ein einziges Ereignis reagieren, nämlich `article-read`, das beim Öffnen eines Artikels die Kennung liefert. Alle anderen Ereignisse, etwa für Likes oder das erste Sichtbarwerden einer Karte, sind optional und dienen der Anbindung eigener Analysewerkzeuge (Anforderung S4). Farben, Schriften und Abstände lassen sich über CSS-Variablen anpassen, einzelne Bereiche einer Karte über Slots durch eigene Inhalte ersetzen (Anforderung S3).

Mit diesem Überblick sind alle Bausteine und ihre Aufgaben eingeführt. Welche Technologien für die einzelnen Bausteine zur Auswahl standen und warum die Entscheidung jeweils so gefallen ist, behandelt das nächste Kapitel.

5 Zusammenfassung

<[Die Zusammenfassung wird erst geschrieben, wenn alle Kapitel fertig sind <== (ca. 1 bis 1,5 Seiten, ohne Abschnitte, mit Ausblick).]>

Glossar

API Application Programming Interface

CMS Content-Management-System

CORS Cross-Origin Resource Sharing

Embedding Darstellung eines Textes als Vektor fester Länge, bei der inhaltlich ähnliche Texte nahe beieinander liegen

GPU Graphics Processing Unit

Host Bestehende Website, in die der Feed als Komponente eingebunden wird

JSON JavaScript Object Notation

LLM Large Language Model

ORM Object-Relational Mapping

REST Representational State Transfer

RSS Really Simple Syndication

SSR Server-Side Rendering

Teaser Kurzfassung eines Artikels mit höchstens drei Sätzen, die auf einer Karte im Feed angezeigt wird

VRAM Video Random Access Memory

Literaturverzeichnis

[1] N. Newman, A. Ross Arguedas *et al.*, „Reuters Institute Digital News Report 2025," Reuters Institute for the Study of Journalism, University of Oxford, Tech. Rep., 2025, letzter Zugriff am 13.10.2026. Online verfügbar: https://reutersinstitute.politics.ox.ac.uk/digital-news-report/2025

[2] T. Zhang, F. Ladhak *et al.*, „Benchmarking Large Language Models for News Summarization," *Transactions of the Association for Computational Linguistics*, Vol. 12, S. 39–57, 2024.

[3] J. Maynez, S. Narayan *et al.*, „On Faithfulness and Factuality in Abstractive Summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, S. 1906–1919.

[4] Inshorts, „Inshorts – News in 60 words," 2026, letzter Zugriff am 13.10.2026. Online verfügbar: https://inshorts.com/

[5] T. Goyal, J. J. Li, und G. Durrett, „News Summarization and Evaluation in the Era of GPT-3," 2022, letzter Zugriff am 13.10.2026. Online verfügbar: https://arxiv.org/abs/2209.12356

[6] Z. Ji, N. Lee *et al.*, „Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, Vol. 55, Nr. 12, S. 1–38, 2023.

[7] Qwen Team, „Qwen3 Technical Report," 2025, letzter Zugriff am 13.10.2026. Online verfügbar: https://arxiv.org/abs/2505.09388

[8] S. Raza und C. Ding, „News Recommender System: A Review of Recent Progress, Challenges and Opportunities," *Artificial Intelligence Review*, Vol. 55, Nr. 1, S. 749–800, 2022.

[9] F. Wu, Y. Qiao *et al.*, „MIND: A Large-scale Dataset for News Recommendation," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, S. 3597–3606.

[10] J. Chen, S. Xiao *et al.*, „BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," 2024, letzter Zugriff am 13.10.2026. Online verfügbar: https://arxiv.org/abs/2402.03216

[11] E. Pariser, *The Filter Bubble: What the Internet Is Hiding from You.* New York: Penguin Press, 2011.

[12] J. Carbonell und J. Goldstein, „The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, S. 335–336.

[13] Z-Image Team, „Z-Image: An Efficient Image Generation Foundation Model with Single-Stream Diffusion Transformer," 2025, letzter Zugriff am 13.10.2026. Online verfügbar: https://arxiv.org/abs/2511.22699

[14] R. T. Fielding, M. Nottingham, und J. Reschke, „HTTP Semantics," Internet Engineering Task Force, RFC 9110, 2022.

Abbildungsverzeichnis

1 Bausteine von wb-newsfeed und ihre Verbindungen 16
2 Ablauf der asynchronen Generierung eines Artikels 18

Tabellenverzeichnis

1 Vergleich verbreiteter Ansätze für Short-Form-Nachrichten 5
2 Technische Rahmenbedingungen laut Briefing 10
3 Auswahl der User Stories je Rolle 11
4 Funktionale Pflichtanforderungen 12
5 Optionale Erweiterungen laut Briefing 14
6 Nicht-funktionale Anforderungen 14
7 Mit generativer KI erstellte Erstfassungen XI

Quellcodeverzeichnis

1 Aufruf des Sprachmodells über Ollama (gekürzt aus `summarizer.ts`) . 19
2 Einbindung des Feeds in eine Seite der Host-Website 21

Anhang

Einsatz generativer KI

Gemäß den Vorgaben der HTL Leonding zum Einsatz generativer KI sind alle Teile dieser Arbeit, deren Erstfassung von einem Sprachmodell erstellt wurde, im Text mit einer Fußnote gekennzeichnet. Tabelle 7 listet diese Teile mit dem verwendeten Werkzeug und dem Link zum Prompt-Verlauf. Alle generierten Inhalte wurden von den Autoren geprüft, mit Quellen belegt und gegebenenfalls korrigiert.

<[Link zum Chat-Verlauf eintragen (Claude Code: Transkript exportieren und z. B. im Repository ablegen) und die letzte Spalte nach der eigenen Überarbeitung anpassen.]> <==

Teil der Arbeit	Werkzeug	Datum	Prompt-Verlauf
Abstract, Zusammenfassung (Kurzfassung)	Claude Opus 5.5	13.10.2026	TODO Link
Kapitel 1 Einleitung	Claude Opus 5.5	13.10.2026	TODO Link
Kapitel 2 Umfeldanalyse	Claude Opus 5.5	13.10.2026	TODO Link
Kapitel 3 Anforderungen	Claude Opus 5.5	13.10.2026	TODO Link
Kapitel 4 Systemüberblick	Claude Opus 5.5	13.10.2026	TODO Link

Tabelle 7: Mit generativer KI erstellte Erstfassungen